

PRODUCT REQUIREMENTS DOCUMENT

Verified Community Directory PWA

Secure MVP Rebuild

A private, verified community networking, communication, and emergency-response platform.

PRIVATE	VERIFIED	MOBILE-FIRST
---------	----------	--------------

Version 1.0 | Draft for implementation planning
Prepared for the Community Directory Project

Document Control

A concise reference for product, design, engineering, and admin stakeholders.

Item	Detail
Document	Product Requirements Document (PRD)
Product	Verified Community Directory PWA
Version	1.0 - Secure MVP Rebuild
Status	Draft for implementation planning
Primary users	Verified community members and authorised administrators
Deployment model	Installable web application (PWA) with secure backend services
Source baseline	Existing static PWA prototype, current system gap analysis, and approved target vision

Purpose of This PRD

This PRD defines the intended product experience, MVP scope, security principles, functional requirements, user journeys, success criteria, and delivery sequence for rebuilding the current community directory prototype into a secure, multi-user system. It is deliberately product-focused: detailed API contracts, database schemas, and test cases belong in the subsequent Software Requirements Specification (SRS).

Decision in one sentence

The existing frontend remains a useful UI reference, but every security-critical capability - member data, identity, device approval, posts, alerts, moderation, and audit history - must move behind authenticated backend services.

Table of Contents

A quick map of the product decisions, requirements, and delivery sequence.

- 1. Product Context and Vision
- 2. Problem Statement
- 3. Product Goals and Non-Goals
- 4. Users, Roles, and Core Jobs
- 5. Product Scope and Feature Priorities
- 6. Product Experience and Key User Journeys
- 7. Security, Privacy, and Trust Principles
- 8. MVP Product Requirements
- 9. Product Data Flow and Operating Model
- 10. Success Metrics and Acceptance Outcomes
- 11. Delivery Roadmap
- 12. Risks, Assumptions, and Open Decisions
- Appendix A. Legacy-to-Target Transition
- Appendix B. Glossary

How to use this document

Read this section when you need to...	Go to
Understand the product purpose, users, and MVP boundary	Sections 1-5
Review expected member/admin experience and trust model	Sections 6-7
Translate the product into buildable requirements and flows	Sections 8-9
Plan delivery, acceptance, and launch decisions	Sections 10-12

1. Product Context and Vision

The Community Directory PWA is intended to be a mobile-first, installable platform that community members can open from their phone home screen without requiring Play Store installation. It should feel like a simple community app while applying the access controls, privacy safeguards, and administrative accountability expected from an official platform.

Product vision

Build a secure, private, administrator-controlled community platform where only approved members can discover authorised profiles, share community updates, raise genuine emergency alerts, and access the system from an approved device.

What the product is

DIRECTORY	COMMUNICATION	TRUST
Find verified community members	Share Inbox updates and Emergency alerts	Use verified accounts and approved devices

Product positioning

It is	It is not
A verified community networking and communication platform	A public contact directory
An installable PWA with a secure server-side system	A static website that trusts browser checks
A shared communication space with admin moderation	A localStorage demo where messages stay on one phone
A privacy-aware directory with controlled profile visibility	A public Google Sheet rendered as an app
A trusted-device workflow using passkeys and approval	An IMEI-tracking application

2. Problem Statement

The current PWA provides a strong visual prototype: member cards, search, filters, profile sheets, Inbox and Emergency screens, PWA metadata, and basic offline shell support. However, it is not suitable for official multi-user deployment because its data and trust model remain browser-side.

Current shortcomings to eliminate

Current behavior	Why it is unsafe / insufficient	Target behavior
Google Sheet is fetched directly by the frontend	The source URL can be discovered and sensitive data can bypass the UI	Backend imports/syncs approved data into a private database and serves protected APIs

Current behavior	Why it is unsafe / insufficient	Target behavior
Member IDs are generated in the browser from name and timestamp	IDs are reproducible and are not authoritative credentials	Backend issues random, unique, non-guessable Member IDs
Member ID typed before posting	Anyone can type another visible ID and impersonate that member	Backend derives author identity from the authenticated session
Inbox and Emergency use localStorage	Posts are not shared, cannot be moderated, and disappear with browser data	Posts are centrally stored, shared, moderated, and auditable
Public Google Form add button	Submission does not equal verified membership	Admin imports/reviews/approves members before activation
No authentication, sessions, or device trust	Anyone who reaches the app can attempt access	OTP, passkeys, trusted devices, roles, and revocation control access

Non-negotiable correction

The frontend is an interface, not a security boundary. All authorisation decisions must be enforced by the backend on every protected request.

3. Product Goals and Non-Goals

Goals

- Give only approved community members secure access to authorised directory information.
- Replace public Google Sheet access with a private, backend-controlled member data flow.
- Make directory search, filters, and profile viewing reliable on mobile and desktop browsers.
- Convert Inbox and Emergency from local demos into shared, centrally stored communication modules.
- Prevent impersonation by deriving identity from secure sessions rather than typed IDs.
- Use passkeys and trusted-device approval instead of impossible IMEI access from a PWA.
- Give admins clear control over members, device changes, moderation, security flags, and audit records.
- Retain the friendly, installable PWA experience of the existing prototype.

Non-goals for the secure MVP

- Native Android/iOS app development, Play Store publishing, or device IMEI tracking.
- Anonymous browsing, public directory access, or self-service activation without approval.
- Private one-to-one chat, voice/video calling, payments, donations, or social-media-style features.
- WebSockets, AI moderation, or automated Google Sheet synchronization before the core workflow is stable.
- SMS OTP as the default identity method; registered email OTP is the practical MVP channel.

4. Users, Roles, and Core Jobs

User / role	Core job	What success looks like
Community Member	Find authorised people, view permitted details, share updates, raise an emergency alert	The app feels simple, trustworthy, and useful without exposing data unnecessarily

User / role	Core job	What success looks like
Administrator	Approve members/devices, moderate posts, resolve emergencies, review incidents	The admin can take accountable action without depending on raw Sheet access or manual browser fixes
Pending Member	Complete identity and device enrolment after approval	They can activate access once, on a supported approved device
Suspended / Deactivated Member	Receive controlled access outcome	They cannot access private data or post until an admin restores status
System Maintainer	Deploy, configure, back up, and troubleshoot safely	Secrets remain outside source code and operational failures are diagnosable

Core jobs-to-be-done

1. When I need to find someone in the community, I want to search and filter verified profiles quickly, so I can connect with the right person.
2. When there is an update, I want to post it once as my verified identity, so every authorised member sees the same record.
3. When there is an urgent issue, I want to raise an alert that is visibly prioritised and traceable, so the community and admins can respond.
4. When I administer the community, I want to approve people and device changes, review suspicious behavior, and moderate content, so the platform remains safe and accountable.

5. Product Scope and Feature Priorities

Priority	Feature set	Why it belongs in this release
Must have	Admin bootstrap, admin login, role control, member import/review/approval, secure Member ID issuance	These establish authoritative records before any user access exists
Must have	Protected directory, search, filters, privacy-based profile views	This replaces the exposed Sheet model with the primary member value
Must have	Email OTP, secure server sessions, passkey/WebAuthn trusted-device enrolment, device-change approval	These make access trustworthy and revocable
Must have	Shared Inbox, Emergency alerts, private media, moderation, alert resolution	These turn visual demo modules into real community functionality
Must have	Security events, flags, audit logs, suspend/revoke actions	Admins need operational visibility and accountability
Should have	Exports, reporting, controlled scheduled data import, content reporting workflow	Useful operational improvements after primary paths are stable
Could have later	SMS OTP, real-time sockets, APK packaging, multilingual UI, notification	Valuable, but not needed to prove the secure MVP

Priority	Feature set	Why it belongs in this release
	integrations	

Feature boundary

Approve member	Verify identity	Approve device	Use directory	Post / alert	Admin monitor
----------------	-----------------	----------------	---------------	--------------	---------------

6. Product Experience and Key User Journeys

6.1 Admin member import and approval

Export CSV	Admin upload	Validate + stage	Review + approve	Secure ID issued	Ready for enrolment
------------	--------------	------------------	------------------	------------------	---------------------

The initial MVP intentionally uses admin-led CSV import rather than live public Sheet access. Google Forms and Sheets may remain a collection source, but not the live production directory source.

6.2 First-time member enrolment

Enter secure Member ID + email	Receive OTP	Verify OTP	Register passkey	Device active	Enter PWA
--------------------------------	-------------	------------	------------------	---------------	-----------

The identity response must remain generic. The system must not reveal whether a particular Member ID, email address, or phone number exists.

6.3 Future login from an approved device

Open PWA	Passkey prompt	User verifies	Backend checks	Session created	Access granted
----------	----------------	---------------	----------------	-----------------	----------------

6.4 New device or reset browser

No trusted device cookie	OTP verification	Device change request	Admin approval	Register new passkey	Old device revoked
--------------------------	------------------	-----------------------	----------------	----------------------	--------------------

OTP alone proves control of a registered contact method. It does not bypass the community policy requiring an administrator-approved device.

6.5 Shared communication

Member experience	Backend behavior	Admin experience
Open Inbox, write an update, add optional media, tap Send	Derive author from session, validate content/media, persist centrally, distribute through feed API	View, remove, or review reports; action is logged
Open Emergency, describe urgency, attach optional media, tap Send	Store as urgent alert with author, timestamp, status, and history	Monitor alert, add resolution note, mark Resolved

7. Security, Privacy, and Trust Principles

Access rule

Protected access is allowed only when the member is approved and active, a valid session exists, a valid WebAuthn assertion is verified, the credential belongs to that member, and an active server-side trusted-device record matches the browser device cookie.

Security principles

Principle	Product implication
Backend is the gatekeeper	All protected API requests validate session, member status, device status, and role; the frontend never grants access by itself.
No ID impersonation	Member IDs identify a record but do not prove identity. The post author comes from the secure session.
OTP abuse control	OTP expiry, attempt caps, resend cooldowns, generic errors, and request-source rate limits are required.
Trusted device, not IMEI	Passkeys plus a server-side device record and HttpOnly cookie provide a web-compatible security model.
Revocation always possible	Admins can suspend accounts, revoke devices, invalidate sessions, remove posts, and resolve alerts.
Auditability	Approval, edit, moderation, device, and suspension actions are recorded with actor, target, time, and outcome.

Privacy model

Data group	Default visibility
Directory card	Verified members: name, photo/initials, city/state, profession or organisation summary
Contact and professional fields	Verified members only where member visibility settings permit
Sensitive personal fields	Admin only by default: date of birth, pincode, family details,

Data group	Default visibility
	spouse/children, caste/sub-caste
Operational security records	Admin only: OTP attempts, device records, security flags, audit logs
Secrets and credentials	Never displayed: OTP values, session tokens, cookie values, credential public-key data, storage secrets

Privacy decisions

- The PWA must not expose a public Google Sheet CSV URL or raw Drive link as the directory data source.
- The PWA must not embed real profile data in JavaScript fallback arrays, service-worker caches, or offline bundles.
- Sensitive fields must be served through role- and visibility-aware backend projections rather than a single full-profile payload.
- Private media must be delivered through authorised time-limited access, not public permanent links.

8. MVP Product Requirements

These requirements describe user-visible product behavior. Detailed interfaces and data schemas should be specified in the SRS.

8.1 Member management

ID	Requirement
PRD-MEM-01	Admins can upload a CSV, validate rows, review issues, and stage records before approval.
PRD-MEM-02	Only approved records receive a backend-generated secure Member ID and become eligible for enrolment.
PRD-MEM-03	Admins can edit, suspend, deactivate, reactivate, and export member records.

8.2 Access and identity

ID	Requirement
PRD-AUTH-01	Unauthenticated users cannot access directory, profile, Inbox, Emergency, or admin data.
PRD-AUTH-02	First-time access requires registered email OTP plus passkey registration and trusted-device activation.
PRD-AUTH-03	Future access from an approved device requires passkey verification and a valid trusted-device record.
PRD-AUTH-04	New-device access requires OTP, a pending device-change request, admin approval, and new passkey registration.

8.3 Directory and profiles

ID	Requirement
PRD-DIR-01	Verified members can search and filter authorised directory fields such as state, blood group, occupation, education sector, and optional approved sub-caste data.
PRD-DIR-02	Profile detail visibility follows role and member privacy settings.
PRD-DIR-03	The app does not fetch Google Sheet CSV data directly and does not deliver unrestricted full-profile dumps to the browser.

8.4 Shared Inbox and Emergency

ID	Requirement
PRD-COM-01	Active verified members can create shared Inbox posts with optional approved media.
PRD-COM-02	Active verified members can create high-priority Emergency alerts with optional approved media.
PRD-COM-03	The sender is automatically shown as the authenticated member; users do not type an ID to post.
PRD-COM-04	Admins can remove inappropriate content, resolve emergency alerts, and see moderation history.

8.5 Security and monitoring

ID	Requirement
PRD-SEC-01	The platform records failed identity checks, wrong OTP attempts, invalid device attempts, and unauthorised post requests.
PRD-SEC-02	The platform rate-limits abuse without allowing an attacker to lock innocent members out.
PRD-SEC-03	Admins can review security events, security flags, device-change requests, and audit logs.

8.6 PWA behavior

ID	Requirement
PRD-PWA-01	The app is installable and mobile-first, with a safe offline app shell.
PRD-PWA-02	The app does not cache private directory or session data as a generic offline snapshot.

ID	Requirement
PRD-PWA-03	The member experience remains simple: clear loading, empty, offline, error, and permission states.

9. Product Data Flow and Operating Model

Target operating model

Form / CSV input	Admin review	Private database	Secure backend APIs	Verified PWA	Admin monitoring
------------------	--------------	------------------	---------------------	--------------	------------------

Google Forms and Google Sheets may remain an administrative intake source. They are not part of the live member-facing data path. The authoritative production record lives in the private database after validation and approval.

Product data flow

Flow	Input	System action	Output
Member import	CSV from admin / Form export	Validate, stage, detect duplicate or incomplete data	Admin review queue
Approval	Admin decision	Generate secure Member ID; assign status PENDING_ENROLLMENT	Eligible member record
Enrolment	Member ID + email + OTP + passkey	Verify identity; register credential/device; create session	Active verified member
Directory search	Query and filter selections	Apply access checks and data visibility projection	Allowed profile cards/details
Inbox/Emergency post	Message and optional media	Derive author, validate, store, expose to approved feed	Shared moderated record
Security monitoring	Failure patterns and admin actions	Log events, apply rate limit, create flags	Admin review actions

Product-state rules

State	Meaning	Member access
STAGED	Imported but not yet approved	No access
PENDING_ENROLLMENT	Approved record awaiting OTP/passkey/device activation	Only enrolment route
ACTIVE	Identity and trusted device are valid	Directory and communication access
SUSPENDED	Temporarily blocked by admin or security action	No protected access

State	Meaning	Member access
DEACTIVATED	Removed from active community platform	No protected access

10. Success Metrics and Acceptance Outcomes

0 Public member-data endpoints	100% Protected post authors derived from session	1 Authoritative private member data source
--	--	--

MVP acceptance outcomes

- An unauthenticated browser receives no directory, profile, Inbox, Emergency, or admin data.
- No public Google Sheet CSV endpoint is used in the running frontend and no real member data is bundled into source code.
- A secure backend-generated Member ID cannot be recreated from a name or timestamp.
- A member cannot post as another member by entering a visible ID.
- An Inbox post and an Emergency alert created by Member A are visible to Member B and an Admin through the central system.
- A new browser/phone remains blocked from private data until its device-change request is approved.
- An admin can approve a member, revoke a device, remove a post, resolve an emergency alert, and review the resulting audit entries.
- The PWA can be installed while private API responses are not blindly stored as offline app data.

Operational product metrics

Metric	MVP target / observation
Directory access without authentication	0 successful requests
Shared post delivery	Post created by one verified member appears in another verified member feed after next refresh cycle
Admin traceability	100% of approvals, device changes, moderation actions, and account status changes have audit records
Device change control	0 new-device activations completed through OTP alone
Privacy hygiene	0 real profiles contained in frontend fallback code or static cache manifests
Usability	Member can complete first enrolment without typing technical concepts such as WebAuthn or cookies

11. Delivery Roadmap

Phase	Focus	Exit criterion
0. Legacy freeze	Preserve UI reference; remove sensitive hardcoded source data; define privacy field matrix	Current prototype is no longer extended as production code
1. Backend foundation	FastAPI, private database, secure configuration, admin bootstrap, audit utility	Backend runs securely with empty database and health checks
2. Admin and member records	Admin authentication, CSV staging, validation, approval, secure ID issuance	Approved member exists in private database with status control
3. Protected directory	React migration, secure directory APIs, filters, privacy projections	Directory UI reads only protected APIs
4. Identity and sessions	Email OTP, rate limits, opaque sessions, logout, security-event recording	No unauthenticated access and no typed-ID impersonation
5. Trusted devices	Passkeys, trusted-device cookie, device change request and approval	New devices are blocked until approved
6. Shared communication	Inbox, Emergency, media storage, moderation, alert resolution	Posts are shared and admin-controlled
7. Security operations	Flags, reporting, export, admin monitoring, hardening	Admin can investigate and act on suspicious behavior
8. PWA hardening and pilot	Safe caching, end-to-end tests, deployment, pilot feedback	Installable secure MVP is ready for controlled use

Build-order rule

Do not start with passkeys or a large admin dashboard inside the legacy single-file HTML. First establish the private database, authoritative member records, protected APIs, and normal secure sessions. Add trusted-device enforcement once the core identity flow exists.

12. Risks, Assumptions, and Open Decisions

Known risks and mitigations

Risk	Mitigation / product decision
Existing public data path may remain discoverable	Remove public Sheet links and real fallback data before any pilot; use backend import only
SMS is costly or unreliable for MVP	Use registered email OTP first; design OTP service as an adapter for later SMS addition
Passkey support varies on old devices	Require supported browsers for protected access; provide admin-assisted device change rather than weaken policy
Lockout rules can be abused to block others	Rate-limit requesting sources and OTP challenges, not arbitrary member accounts

Risk	Mitigation / product decision
Sensitive fields can be overshared by default	Publish and enforce a signed-off privacy field matrix before importing real data
Large media affects cost and performance	Apply file type/size limits and private storage lifecycle rules
Legacy UI is monolithic	Treat it as a visual reference and rebuild modularly; do not patch the single file into a large production app

Open decisions required before pilot

ID	Decision
OD-01	Which administrators can approve members, device changes, and emergency resolution?
OD-02	Which specific profile fields are visible to verified members, hidden by default, or admin-only?
OD-03	What email provider and sender identity will dispatch OTPs in production?
OD-04	What media size, retention period, and deletion policy are acceptable for the community?
OD-05	What is the community's escalation process when an Emergency alert is raised?
OD-06	What legal/privacy notice and consent wording are required before member data is imported?

Appendix A. Legacy-to-Target Transition

Legacy prototype behavior	Target secure behavior
Single static `index.html` includes UI, data fetching, filters, messaging, and trust checks	Modular React PWA frontend plus FastAPI backend, clearly separated services and admin routes
Google Sheet CSV fetch from browser	Admin import/sync into private database; secure server-side APIs
Fallback array contains member data	Safe empty state or fake development fixtures only
Client-side `makeId(name + timestamp)`	Secure random backend-issued Member ID
Browser `MEMBER_BY_ID` object validates posts	Backend session identifies author and validates permission
localStorage Inbox/Emergency	Central `posts` store with shared feed, moderation, and history
Public Google Form Add Members button	Admin import/approval workflow or separately moderated membership request
Public Drive thumbnails	Private media storage and signed access paths
Static service-worker asset cache	Safe app-shell cache with no broad caching of private data

Explicitly prohibited in the final product

- Public Google Sheet CSV as the live directory source.
- Real community member data embedded in frontend code, fallback arrays, or static caches.
- Browser-generated Member IDs or Member IDs treated as passwords.
- Member ID fields that allow a person to post as someone else.
- localStorage as the source of truth for shared Inbox or Emergency messages.
- OTP-only activation for a new or reset device.
- IMEI-based device verification for the PWA.
- Admin changes without audit records.

Appendix B. Glossary

Term	Meaning
PWA	Progressive Web Application. An installable web app that can open like an app from a phone home screen.
Member ID	A secure, random, backend-issued identifier assigned to an approved member. It is not a password or proof of identity.
OTP	One-Time Password sent to a registered contact method for short-lived verification.
WebAuthn / passkey	Browser-supported public-key authentication using a device biometric, PIN, or screen lock.
Trusted device	A server-approved device/browser record linked to a passkey and secure device cookie.
Opaque session	A random server-recognised session identifier stored in a secure cookie; the browser does not hold user claims or secrets.
Security event	A recorded failed or suspicious action, such as incorrect OTPs or invalid device attempts.
Security flag	An admin-reviewable issue created when security events meet a configured threshold.
Audit log	An immutable-style record of an important action: who acted, what changed, when, and why.
Privacy projection	A backend-controlled subset of profile fields returned based on the viewer role and member visibility settings.

End of Product Requirements Document